Activities Google Chrome Oct 11 11:37 65 %

COM DOC Revision Suki Software Engineer H HackerRank

hackerrank.com/test/1sf845sps94/questions/e8qb5mk89j3

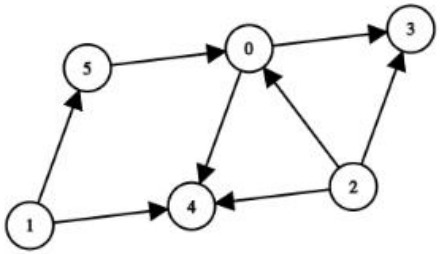
Suki Software Engineer Hiring Test - Roorkee Answered: **4 / 12** **01 hour 23 mins** Mohammad Nazim

ALL

1 2 3 4 5 6 7

3. Topological Order

Which of the following is the correct topological order for the given graph?



Pick **ONE OR MORE** options

- [ ] 2 1 5 3 4 0
- [ ] 5 2 1 0 3 4

Activities Google Chrome Oct 11 11:37 65 %

COM DOC Revision Suki Software Engineer H HackerRank

hackerrank.com/test/1sf845sps94/questions/e8qb5mk89j3

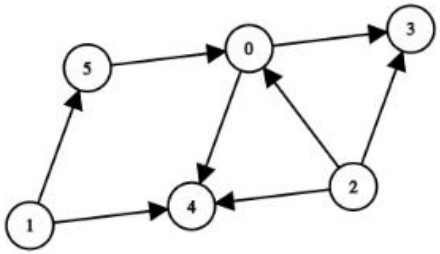
Suki Software Engineer Hiring Test - Roorkee Answered: **4 / 12** **01 hour 23 mins** Mohammad Nazim

ALL

1 2 3 4 5 6 7

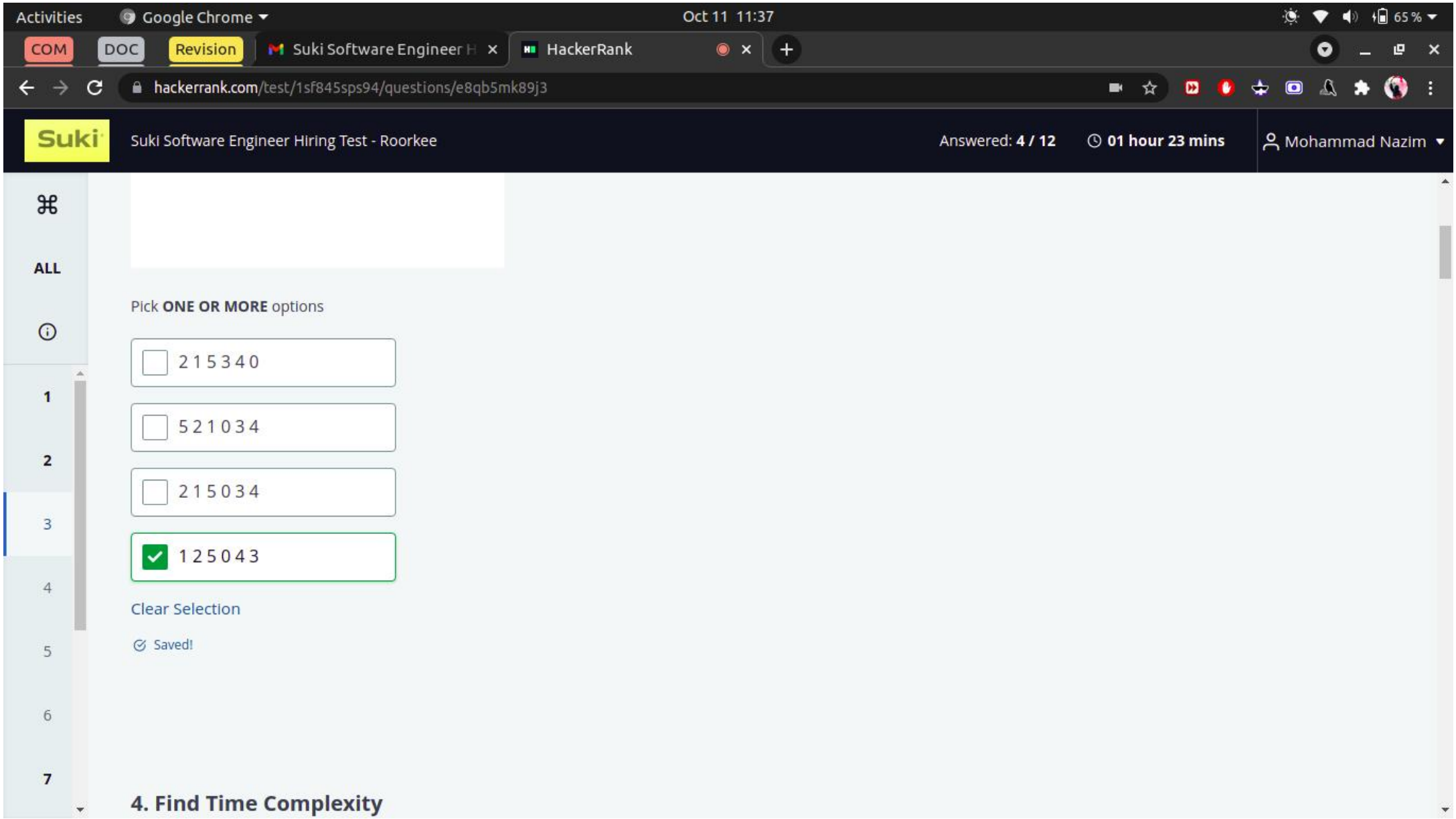
3. Topological Order

Which of the following is the correct topological order for the given graph?



Pick **ONE OR MORE** options

- [ ] 2 1 5 3 4 0
- [ ] 5 2 1 0 3 4


Activities
Google Chrome
Oct 11 11:37
65 %
COM
DOC
Revision
Suki Software Engineer H
HackerRank
hackerrank.com/test/1sf845sps94/questions/e8qb5mk89j3
Suki
Suki Software Engineer Hiring Test - Roorkee
Answered: 4 / 12
01 hour 23 mins
Mohammad Nazim
ALL
Pick ONE OR MORE options
215340
521034
215034
125043
Clear Selection
Saved!
1
2
3
4
5
6
7
4. Find Time Complexity

⌘
ALL
ⓘ
1
2
3
4
5
6
7

4. Find Time Complexity

What will be the time complexity of the following code?

```
def countValues(n):
    count = 0
    for i = 1; i <= n; i += 1:
        for j = i; j <= n; j += i:
            count += 1
    return count
```

Pick **ONE** option

- ○ O(log n)
- ○ O(n)
- ◉ O(nlog n)

Activities Google Chrome Oct 11 11:37 65 %

COM DOC Revision Suki Software Engineer H HackerRank

hackerrank.com/test/1sf845sps94/questions/1078mf5qo

Suki Software Engineer Hiring Test - Roorkee Answered: **4 / 12** **01 hour 23 mins** Mohammad Nazim

ALL

2 3 4 5 6 7 8

5. If n & (n - 1) == 0, what can you say about integer n?

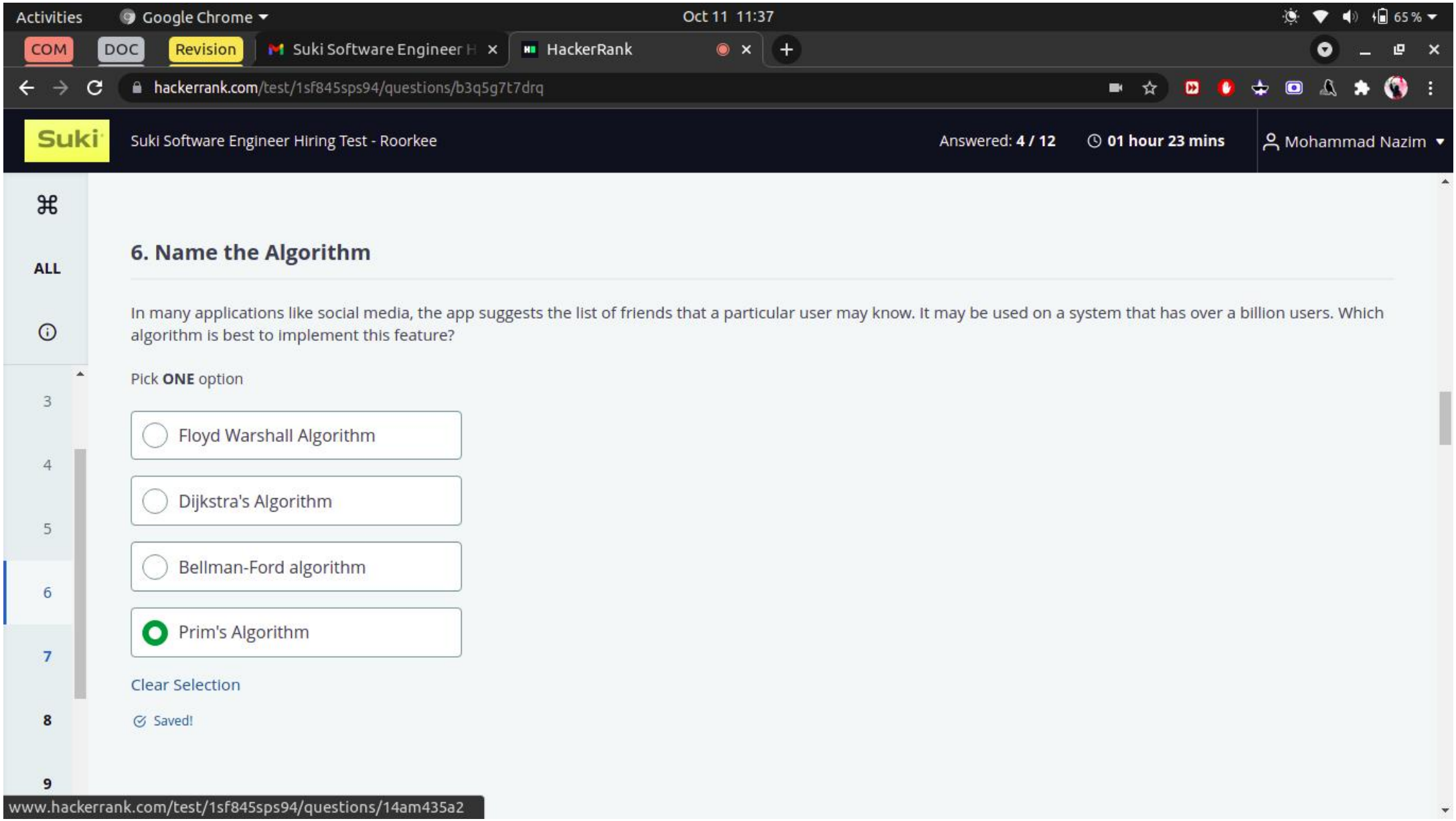
If n & (n - 1) == 0, what can you say about integer n?

Pick **ONE** option

- ○ n is an even number.
- ◉ n is a power of 2.
- ○ n is an odd number.
- ○ n is not a power of 2.

Clear Selection

Saved!


Activities
Google Chrome
Oct 11 11:37
65 %
COM
DOC
Revision
Suki Software Engineer H
HackerRank
hackerrank.com/test/1sf845sps94/questions/b3q5g7t7drq
Suki
Suki Software Engineer Hiring Test - Roorkee
Answered: 4 / 12
01 hour 23 mins
Mohammad Nazim
ALL
6. Name the Algorithm
In many applications like social media, the app suggests the list of friends that a particular user may know. It may be used on a system that has over a billion users. Which algorithm is best to implement this feature?
Pick ONE option
Floyd Warshall Algorithm
Dijkstra's Algorithm
Bellman-Ford algorithm
Prim's Algorithm
Clear Selection
Saved!
3
4
5
6
7
8
9
www.hackerrank.com/test/1sf845sps94/questions/14am435a2

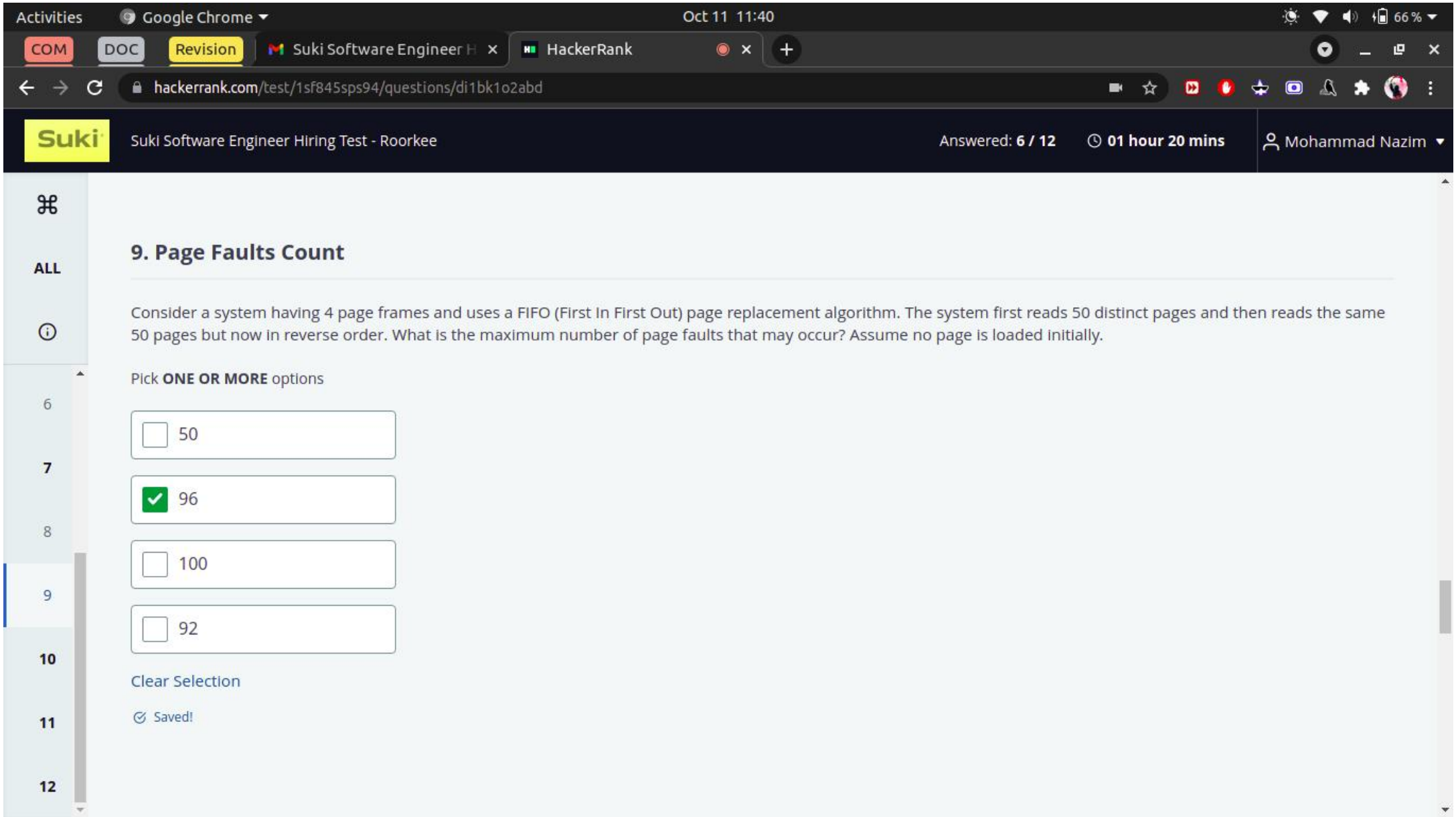
What is the running time of the following algorithm:

```
Procedure A(n)
begin
    if (n < 3)
        return 1
    else
        return A(ceiling(sqrt(n)))
    end
end
```

is best described by

Pick **ONE** option

- ○ O (n)
- ◉ O (log n)
- ○ O (log log n)
- ○ O (1)


Activities
Google Chrome
Oct 11 11:40
66 %
COM
DOC
Revision
Suki Software Engineer H
HackerRank
hackerrank.com/test/1sf845sps94/questions/di1bk1o2abd
Suki
Suki Software Engineer Hiring Test - Roorkee
Answered: 6 / 12
01 hour 20 mins
Mohammad Nazim
ALL
9. Page Faults Count
Consider a system having 4 page frames and uses a FIFO (First In First Out) page replacement algorithm. The system first reads 50 distinct pages and then reads the same 50 pages but now in reverse order. What is the maximum number of page faults that may occur? Assume no page is loaded initially.
Pick ONE OR MORE options
50
96
100
92
Clear Selection
Saved!
6
7
8
9
10
11
12

Activities Google Chrome Oct 11 11:40 66 %

COM DOC Revision Suki Software Engineer H HackerRank

hackerrank.com/test/1sf845sps94/questions/e75dln44

Suki Software Engineer Hiring Test - Roorkee Answered: **6 / 12** **01 hour 20 mins** Mohammad Nazim

ALL

10. Which is the right answer to the following?

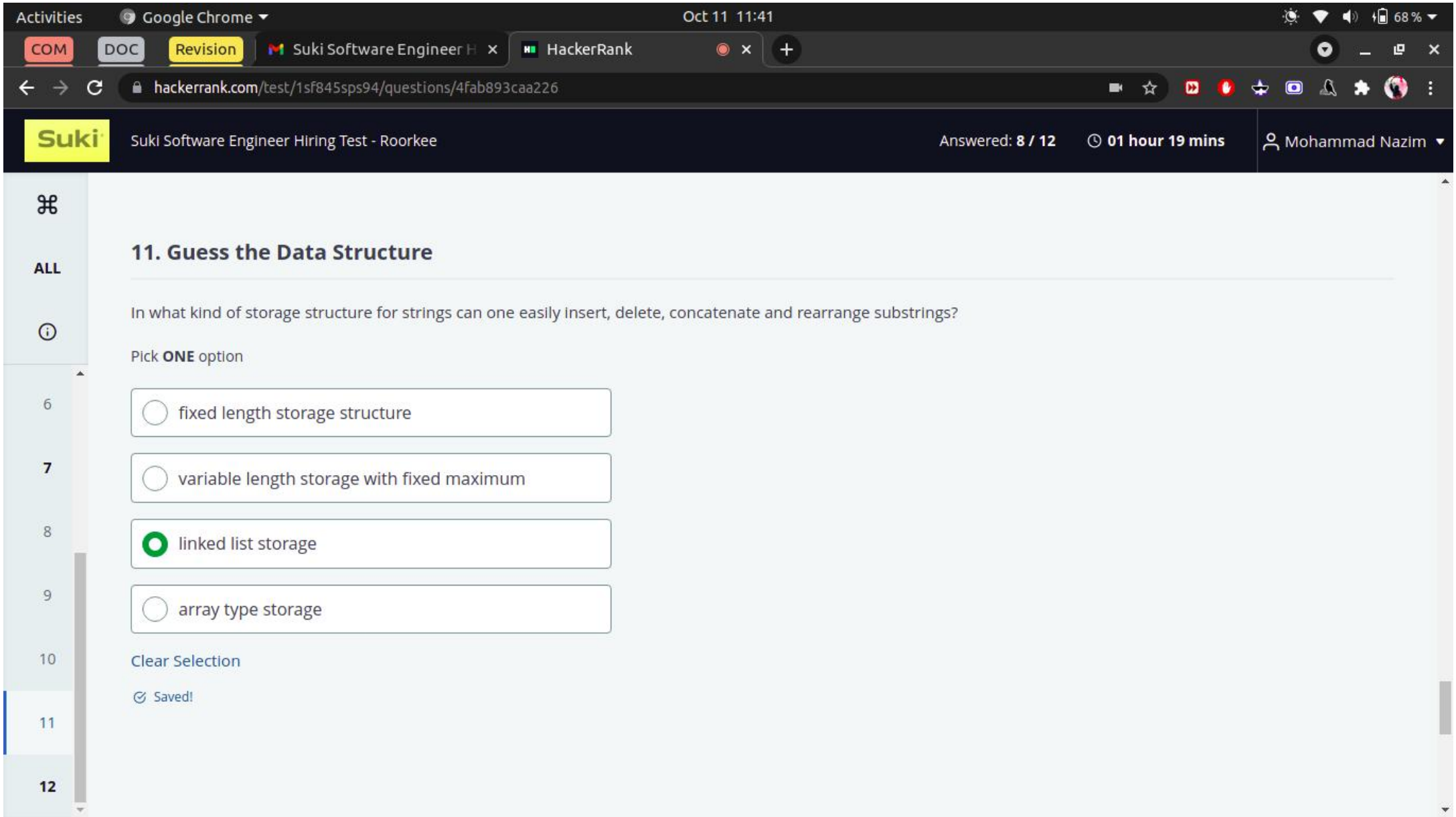
B - Trees are used commonly for:

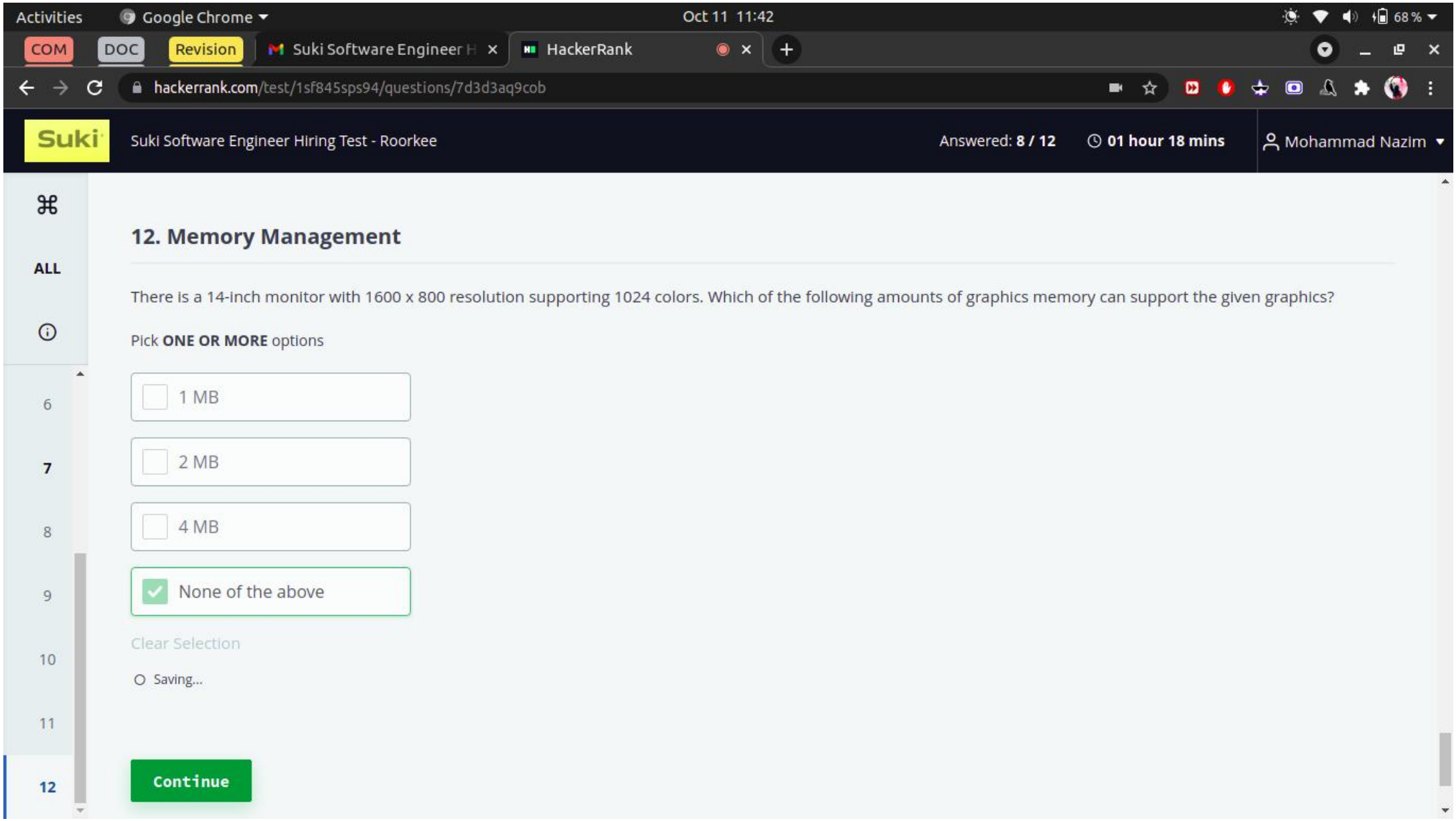
Pick **ONE** option

- efficiently sorting very large files
- for checking the consistency of relational databases
- speeding up database search, delete, and insert operations
- organizing hash tables efficiently

Clear Selection

6 7 8 9 10 11 12


Activities
Google Chrome
Oct 11 11:41
68 %
COM
DOC
Revision
Suki Software Engineer H
HackerRank
hackerrank.com/test/1sf845sps94/questions/4fab893caa226
Suki
Suki Software Engineer Hiring Test - Roorkee
Answered: 8 / 12
01 hour 19 mins
Mohammad Nazim
ALL
11. Guess the Data Structure
In what kind of storage structure for strings can one easily insert, delete, concatenate and rearrange substrings?
Pick ONE option
fixed length storage structure
variable length storage with fixed maximum
linked list storage
array type storage
Clear Selection
Saved!
6
7
8
9
10
11
12


Activities
Google Chrome
Oct 11 11:42
68 %
COM
DOC
Revision
Suki Software Engineer H
HackerRank
hackerrank.com/test/1sf845sps94/questions/7d3d3aq9cob
Suki
Suki Software Engineer Hiring Test - Roorkee
Answered: 8 / 12
01 hour 18 mins
Mohammad Nazim
ALL
12. Memory Management
There is a 14-inch monitor with 1600 x 800 resolution supporting 1024 colors. Which of the following amounts of graphics memory can support the given graphics?
Pick ONE OR MORE options
1 MB
2 MB
4 MB
None of the above
Clear Selection
Saving...
6
7
8
9
10
11
12
Continue

Activities Google Chrome Oct 11 12:17 91 %

COM DOC Revision Suki Software Engineer H HackerRank

hackerrank.com/test/1sf845sps94/questions/e59dgtam1e0

43m left

ALL

1 2 3 4 5 6 7

1. Custom-Sorted Array

In an array, elements at any two indices can be swapped in a single operation called a *move*. For example, if the array is *arr = [17, 4, 8]*, swap *arr[0] = 17* and *arr[2] = 8* to get *arr' = [8, 4, 17]* in a single move. Determine the minimum number of moves required to sort an array such that all of the *even* elements are at the beginning of the array and all of the *odd* elements are at the end of the array.

Example

arr = [6, 3, 4, 5]

The following four arrays are valid custom-sorted arrays:

- *a = [6, 4, 3, 5]*
- *a = [4, 6, 3, 5]*
- *a = [6, 4, 5, 3]*
- *a = [4, 6, 5, 3]*

The most efficient sorting requires *1* move: swap the *4* and the *3*.

Function Description

Complete the function *moves* in the editor below.

moves has the following parameter(s):

int arr[n]: an array of positive integers

Returns

Language C++

Autocomplete Ready

```
/*
 * Complete the 'moves' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER_ARRAY arr as
parameter.
 */
void swap(vector<int> &arr, int i, int j)
{
    int temp=arr[i];
    arr[i]=arr[j];
    arr[j]=temp;
}
int moves(vector<int> arr) {
    int n=arr.size();
    int firstOdd=0, lastEven=n-1;
    while(firstOdd<n && arr[firstOdd]%2==0) firstOdd+
+;
    if(firstOdd==n) return 0;
    while(lastEven>=0 && arr[lastEven]%2) lastEven--;
    if(lastEven<0) return -1;
    if(lastEven<firstOdd) return 0;

    int ans=0;
    while(firstOdd<lastEven)
    {
        ans++;
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

- *a = [4, 6, 5, 3]*

The most efficient sorting requires *1* move: swap the *4* and the *3*.

Function Description

Complete the function *moves* in the editor below.

moves has the following parameter(s):

int arr[n]: an array of positive integers

Returns

int: the minimum number of moves it takes to sort an array of integers with all even elements at earlier indices than any odd element

Note: The order of the elements within even or odd does not matter.

Constraints

- $2 \le n \le 10^5$
- $1 \le arr[i] \le 10^9$, where $0 \le i < n$.
- It is guaranteed that array *a* contains at least one *even* and one *odd* element.

▶ **Input Format for Custom Testing**

▼ **Sample Case 0**

Sample Input 0

```
STDIN   Function
-----   --------
```

Language C++

Autocomplete Ready

```
/*
 * Complete the 'moves' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER_ARRAY arr as
parameter.
 */
void swap(vector<int> &arr, int i, int j)
{
    int temp=arr[i];
    arr[i]=arr[j];
    arr[j]=temp;
}
int moves(vector<int> arr) {
    int n=arr.size();
    int firstOdd=0, lastEven=n-1;
    while(firstOdd<n && arr[firstOdd]%2==0) firstOdd+
+;
    if(firstOdd==n) return 0;
    while(lastEven>=0 && arr[lastEven]%2) lastEven--;
    if(lastEven<0) return -1;
    if(lastEven<firstOdd) return 0;

    int ans=0;
    while(firstOdd<lastEven)
    {
        ans++;
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

43m left

ALL

- $2 \le n \le 10^5$
- $1 \le arr[i] \le 10^9$, where $0 \le i < n$.
- It is guaranteed that array *a* contains at least one *even* and one *odd* element.

▶ **Input Format for Custom Testing**

▼ **Sample Case 0**

Sample Input 0

```
STDIN    Function
-----    -------
4    →   arr[] size n = 4
13   →   arr = [13, 10, 21, 20]
10
21
20
```

Sample Output 0

```
1
```

Explanation 0

Swap *arr[0]* and *arr[3]* to get the custom-sorted array *arr' = [20, 10, 21, 13]* in 1 move.

▶ **Sample Case 1**

Language C++

Autocomplete Ready

```
/*
 * Complete the 'moves' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER_ARRAY arr as
parameter.
 */
void swap(vector<int> &arr, int i, int j)
{
    int temp=arr[i];
    arr[i]=arr[j];
    arr[j]=temp;
}
int moves(vector<int> arr) {
    int n=arr.size();
    int firstOdd=0, lastEven=n-1;
    while(firstOdd<n && arr[firstOdd]%2==0) firstOdd+
+;
    if(firstOdd==n) return 0;
    while(lastEven>=0 && arr[lastEven]%2) lastEven--;
    if(lastEven<0) return -1;
    if(lastEven<firstOdd) return 0;

    int ans=0;
    while(firstOdd<lastEven)
    {
        ans++;
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

Activities Google Chrome Oct 11 12:17 91 %

COM DOC Revision Suki Software Engineer H HackerRank

hackerrank.com/test/1sf845sps94/questions/3ak2jrrhj0a

43m left

2. Monsoon Umbrellas

Umbrellas are available in different sizes that are each able to shelter a certain number of people. Given the number of people needing shelter and a list of umbrella sizes, determine the minimum number of umbrellas necessary to cover exactly the number of people given and no more. If there is no combination of umbrellas of the same or different sizes that covers exactly that number of people, return -1.

Example
requirement = 5
sizes = [3, 5]
One umbrella can cover exactly *5* people, so the function should return *1*.

Example
requirement = 8
sizes = [3, 5]
It is possible to use *1* umbrella of size *3* and *1* umbrella of size *5* to cover exactly *8* people, so the function should return *2*.

Example
requirement = 7
sizes = [3, 5]
There is no combination of umbrellas that cover exactly *7* people, so the function should return *-1*.

Function Description

Language C++

Autocomplete Ready

```
/*
 * Complete the 'getUmbrellas' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts following parameters:
 *  1. INTEGER requirement
 *  2. INTEGER_ARRAY sizes
 */
void solve(vector<int> &sizes, int s, int count, int n, int &ans, vector<vector<int>> &dp)
{
    if(n<0) return;
    if(n==0){
        if(n==0)    ans=min(ans, count);
        return;
    }
    if(s==sizes.size()) return;
    if(dp[s][n]!=-1) {
        ans = min(ans, count + dp[s][n]);
        return;
    }
    solve(sizes, s+1, count+1, n-sizes[s], ans, dp);
    solve(sizes, s+1, count, n, ans, dp);
}
int getUmbrellas(int requirement, vector<int> sizes) {
    vector<vector<int>> dp(requirement+1, vector<int>(sizes.size()+1, 10000));
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

43m left

One umbrella can cover exactly *5* people, so the function should return *1*.

Example

requirement = 8

sizes = [3, 5]

It is possible to use *1* umbrella of size *3* and *1* umbrella of size *5* to cover exactly *8* people, so the function should return *2*.

Example

requirement = 7

sizes = [3, 5]

There is no combination of umbrellas that cover exactly *7* people, so the function should return *-1*.

Function Description

Complete the function *getUmbrellas* in the editor below.

getUmbrellas has the following parameter(s):

int requirement: the number of people to shelter

int sizes[n]: an array of umbrella sizes available

Returns:

int: the minimum number of umbrellas required to cover exactly requirement people, or -1 if it is impossible

Constraints

- *1 ≤ requirement, m, sizes[i] ≤ 1000*

▶ **Input Format for Custom Testing**

Language C++

Autocomplete Ready

```
/*
 * Complete the 'getUmbrellas' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts following parameters:
 *  1. INTEGER requirement
 *  2. INTEGER_ARRAY sizes
 */
void solve(vector<int> &sizes, int s, int count, int n, int &ans, vector<vector<int>> &dp)
{
    if(n<0) return;
    if(n==0){
        if(n==0)    ans=min(ans, count);
        return;
    }
    if(s==sizes.size()) return;
    if(dp[s][n]!=-1) {
        ans = min(ans, count + dp[s][n]);
        return;
    }
    solve(sizes, s+1, count+1, n-sizes[s], ans, dp);
    solve(sizes, s+1, count, n, ans, dp);
}
int getUmbrellas(int requirement, vector<int> sizes) {
    vector<vector<int>> dp(requirement+1, vector<int>(sizes.size()+1, 10000));
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

42m left

- *1 ≤ requirement, m, sizes[i] ≤ 1000*

▶ Input Format for Custom Testing

▼ Sample Case 0

Sample Input 0

```
STDIN       Function
-----       --------
4       →   requirement = 4
2       →   sizes[] size n = 2
2       →   sizes = [2, 4]
4
```

Sample Output 0

```
1
```

Explanation 0

The first size of umbrella can cover *sizes[0] = 2* people, and the second can cover *sizes[1] = 4* people. To cover exactly *requirement = 4* people, use either *2* umbrellas of type *sizes[0]* or *1* umbrella of type *sizes[1]*. The minimal number is *1*.

▶ Sample Case 1

Language C++

Autocomplete Ready

```
/*
 * Complete the 'getUmbrellas' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts following parameters:
 *  1. INTEGER requirement
 *  2. INTEGER_ARRAY sizes
 */
void solve(vector<int> &sizes, int s, int count, int
n, int &ans, vector<vector<int>> &dp)
{
    if(n<0) return;
    if(n==0){
        if(n==0)    ans=min(ans, count);
        return;
    }
    if(s==sizes.size()) return;
    if(dp[s][n]!=-1) {
        ans = min(ans, count + dp[s][n]);
        return;
    }
    solve(sizes, s+1, count+1, n-sizes[s], ans, dp);
    solve(sizes, s+1, count, n, ans, dp);
}
int getUmbrellas(int requirement, vector<int> sizes) {
    vector<vector<int>> dp(requirement+1, vector<int>
(sizes.size()+1, 10000));
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

42m left

Sample Output 0

```
1
```

Explanation 0

The first size of umbrella can cover *sizes[0] = 2* people, and the second can cover *sizes[1] = 4* people. To cover exactly *requirement = 4* people, use either *2* umbrellas of type *sizes[0]* or *1* umbrella of type *sizes[1]*. The minimal number is *1*.

▼ Sample Case 1

Sample Input 1

```
STDIN       Function
-----       --------
1       →   requirement = 1
1       →   sizes[] size n = 1
2       →   sizes = [2]
```

Sample Output 1

```
-1
```

Explanation 1

There is *requirement = 1* person to cover and *n = 1* umbrella size that covers *sizes[0] = 2* people. Because there is no way to protect exactly one person, the returned value is *-1*.

Language C++

Autocomplete Ready

```
/*
 * Complete the 'getUmbrellas' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts following parameters:
 *  1. INTEGER requirement
 *  2. INTEGER_ARRAY sizes
 */
void solve(vector<int> &sizes, int s, int count, int n, int &ans, vector<vector<int>> &dp)
{
    if(n<0) return;
    if(n==0){
        if(n==0)    ans=min(ans, count);
        return;
    }
    if(s==sizes.size()) return;
    if(dp[s][n]!=-1) {
        ans = min(ans, count + dp[s][n]);
        return;
    }
    solve(sizes, s+1, count+1, n-sizes[s], ans, dp);
    solve(sizes, s+1, count, n, ans, dp);
}
int getUmbrellas(int requirement, vector<int> sizes) {
    vector<vector<int>> dp(requirement+1, vector<int>(sizes.size()+1, 10000));
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit